

ig-notion-sync — full setup guide

A tool that pulls your Instagram post metrics (reach, views, saves, shares, watch time) into a Notion database. Idempotent — re-running just refreshes the numbers, it never duplicates rows.

This guide walks you through the entire setup from zero. **About 45 minutes the first time** if everything goes smoothly. Most of that is clicking through Meta's developer dashboard, which is the same painful path for everyone.

You don't need any coding experience. This guide assumes you've never opened a terminal, never installed Python, never used GitHub. We'll explain each piece the first time it shows up.

How this guide works

There are six real steps:

1. **Set up your Notion database** — create the spreadsheet-like database in Notion that will hold your post metrics.
2. **Set up Instagram access** — create a small "app" inside Meta's developer site that gives this tool permission to read your Instagram metrics.
3. **Install the tool on your computer** — download the code, install the plumbing it needs, and tell it about your Notion + Instagram accounts.
4. **Run your first sync** — pull your real posts in.
5. **Keep it running** — refresh the access token every two months and (optionally) schedule it to run automatically.
6. **Ask Claude about your analytics** — once the data's flowing, use the Claude chat app to read the database and answer questions about your performance in plain English.

Before all of that, there's a short **Step 0** where you install Claude Code if you want the easy path.

Two ways to follow each step

Throughout the guide, anywhere you'd normally run a terminal command you'll see two options:

 **With Claude Code (recommended if you're not technical):** paste a prompt into Claude and it runs the command for you. You'll just answer questions in plain English.

 **Manual (terminal):** copy-paste the command into your terminal yourself.

Pick whichever path you prefer for each step — you can mix and match. Browser steps (Notion, Meta's developer dashboard) you'll always do yourself; no amount of automation removes those.

Before you start — what you'll need

You need all of these. None of them can be skipped or automated, because they each live inside someone else's website (Instagram, Facebook, Notion).

- **An Instagram Business or Creator account.** Personal accounts can't see the metrics this tool needs. In the Instagram app: Settings → Account type and tools → Switch to professional. (You can switch back any time.)
- **That Instagram account linked to a Facebook Page.** Instagram requires this — the metrics actually come through Facebook's developer API. In the Instagram app: Settings → Account Center → Accounts → link a Facebook Page. (The Facebook Page can be empty; it just has to exist.)
- **A Notion workspace you own.** The free plan is fine. If you don't have one, go to notion.so and sign up first.
- **A Mac, a Linux machine, or a Windows machine with WSL.** WSL is Windows' built-in Linux mode — if you're on Windows and don't already have it, search "install WSL" and follow Microsoft's official guide before continuing.
- **About 45 minutes of uninterrupted time** for the first setup.

You do *not* need to know how to code, have Python installed, or know what a terminal is. We'll handle all of that below.

Step 0 — Install Claude Code (the helper)

What you're doing and why

Claude Code is a free app from Anthropic that lets you talk to Claude (an AI assistant) inside your terminal. Throughout this guide, anywhere there's a command to run, you can paste a plain-English prompt to Claude and it will run the command for you, explain what happened, and fix problems as they come up. If you skip this, you'll need to run terminal commands manually.

Strongly recommended for non-technical users. This is the difference between "I have to figure out what `cd` means" and "I just answer questions."

How to install it

1. Go to <https://docs.claude.com/en/docs/claude-code/quickstart>
2. Follow the install instructions on that page (it's a one-line install on Mac and Linux; Windows users use it from inside WSL).
3. The first time you run `claude` in a terminal, it'll ask you to sign in with an Anthropic account. Free signup at claude.com.

How to "open Claude Code" later

Whenever this guide says "**open Claude Code**," here's what that means:

On Mac:

1. Press **Cmd + Space** to open Spotlight, type **terminal**, hit Enter.
2. A black or white window appears with text. That's your terminal.
3. Make a folder for code if you don't have one yet — type this and hit Enter:

```
mkdir -p ~/Documents/Code && cd ~/Documents/Code
```

4. Type **claude** and hit Enter. Claude Code starts inside that folder.

On Linux / Windows (WSL):

1. Open your terminal app (Linux: search "Terminal"; Windows: search "WSL").
2. Type **mkdir -p ~/Code && cd ~/Code** and hit Enter (only the first time).
3. Type **claude** and hit Enter.

Once Claude Code is running, you'll see a prompt where you can type or paste messages. That's where the  prompts in this guide go.

Step 1 — Notion database

What you're doing and why

The tool needs somewhere to put your Instagram metrics. That somewhere is a **Notion database** — think of it as a spreadsheet where each row is one Instagram post and each column is a piece of information about it (caption, likes, reach, etc.).

We also need to give the tool **permission** to write to that database. Notion calls this an "integration" — basically a username and password the tool uses to log in to your Notion workspace. You'll create the integration, copy its secret key, and connect it to the database.

By the end of this step you'll have **two values written down**:

- A long secret key starting with **ntn_...** (the Notion integration secret)
- A link to your database (which we'll turn into an ID later)

1a. Get the database

Option A — Duplicate the template (recommended):

1. Open the template from Creator College:
<https://defiant-corn-5ee.notion.site/Creator-College-VIP-2d8c1d7a2fc5800887f8fabdb1b49ba?pvs=143>

The walkthrough video is here:

<https://content.creatorcollege.com/content/01KQZ1S6GHE64P2APNN26P4SK5/sections/01KQZ3GF2WM7B83VXMF96FT1PV/content/01KQZ3GF2X4JS9CX1NEVBREFY8>

- In the top-right corner of the page, click **Duplicate** → choose your workspace.
- Open the new database in your workspace.
- Check the columns against the schema table in Option B below.** The template may predate recent script changes — if any column from that table is missing, add it now with the type listed. In particular, confirm **New Followers** and **Skip Rate (%)** exist (both Number); these were added to the script after the template was published, so older copies won't have them. The script can't create columns — if one is missing and Instagram returns a value for it, the row write fails with **validation_error**.

Option B — Build it manually:

If the template link breaks or you'd rather build from scratch, create a new database (full-page works best) and add these columns **exactly as named** — the script matches by column name and will fail loudly if anything is missing or misspelled.

Column name	Type	Notes
Name	Title	Manual — never overwritten
IG Media ID	Text	Filled on first sync
Link to post	URL	Filled on first sync
Status	Status	Add options: Idea , Ideas for later , Draft , Editing , Record , Done
Topics	Select	Manual — add your own options
Mission	(any)	Manual — never overwritten
Intensity	(any)	Manual — never overwritten
Inspo	(any)	Manual — never overwritten
Platform	Select	Add options: Instagram , TikTok , YouTube
Publication Date	Date	
Caption	Text	
Thumbnail	URL	
Media Type	Select	Add options: Reel , Image , Carousel
Last Synced	Date	
Likes	Number	

Column name	Type	Notes
Comments	Number	
Reach	Number	
Total views	Number	
Saves	Number	
Shares	Number	
Average Watch Time (s)	Number	Reels only
Total Watch Time (s)	Number	Reels only
New Followers	Number	Optional. IG's <code>follows</code> per-media metric — many accounts get blank.
Skip Rate (%)	Number	Optional, Reels only. Needs Graph API <code>v22.0+</code> in <code>.env</code> to populate.

The two **Optional** columns above are safe to omit if you don't care. Include them only if you want the script to fill them when IG returns a value (most accounts on default settings won't). If a column is missing and IG *does* return the metric, the Notion write fails with `validation_error` — so either add the column or don't enable the metric.

1b. Create a Notion integration

An "integration" is the digital key the tool uses to log in to your Notion account. You create it once and keep the secret somewhere safe.

- Go to <https://www.notion.so/profile/integrations> (sign in if asked).
- Click **+ New integration**:
 - Name:** `ig-notion-sync` (anything works — this is just so you can recognize it later).
 - Workspace:** your workspace.
 - Type:** **Internal**. (Internal means only you can use it.)
- Click **Save**, then open the integration's **Configuration** tab.
- Under **Capabilities**, enable: **Read content**, **Update content**, **Insert content**. **Disable user info** (the tool doesn't need it).
- Copy the **Internal Integration Secret** (starts with `ntn_` or `secret_`). **Save it now in a notes app — Notion only shows it once.** This is what the tool will use as `NOTION_TOKEN`.

1c. Connect the integration to your database

Creating the integration isn't enough — you also have to explicitly invite it to your database. (Notion does this for privacy: by default, integrations

can't see anything.)

1. Open your database in Notion (as a full page, not embedded).
2. Click the ... menu in the top-right corner → **Connections** → search for `ig-notion-sync` → **Confirm**.

Skipping this step is the #1 cause of `object_not_found` errors later.

If the sync says it can't find your database, come back here.

1d. Get the database ID

The tool needs to know *which* database to write to. The ID is hidden inside the database's URL.

1. With the database open as a full page, copy the URL from your browser's address bar. It looks like:
`https://www.notion.so/<workspace>/<32-character-id>?v=<view-id>`
2. The 32-character chunk **before** `?v=` is your database ID. The tool needs it reformatted with dashes:
`xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx` (insert dashes at positions 8, 12, 16, 20). This is what the tool will use as `NOTION_DATA_SOURCE_ID`.

 **Don't worry about the formatting yourself** — in Step 3, Claude will reformat the URL for you. You just need to paste it in.

You should now have two values saved somewhere safe:

- `NOTION_TOKEN` — the `ntn_...` secret from 1b
- The database URL (Claude will reformat it) **or** the dashed `NOTION_DATA_SOURCE_ID`

Step 2 — Meta developer app & Instagram token

What you're doing and why

Instagram itself doesn't have a public way for tools to read your post metrics. **Facebook does** — and since Meta owns both, your Instagram metrics flow through Facebook's developer API.

To use that API, you have to register a small "app" on Meta's developer site. This app is just paperwork: it doesn't do anything on its own, doesn't publish posts, and isn't visible to anyone but you. Its only purpose is to **give this tool permission to read your Instagram metrics on your behalf**.

Once the app exists, Meta hands you a few values you'll save and paste into the tool:

- `META_APP_ID` and `META_APP_SECRET` — IDs for the app you just created
- `IG_ACCESS_TOKEN` — the actual permission slip, good for 60 days
- `IG_USER_ID` — your Instagram account's numeric ID

This is the longest step. Take your time, and don't worry if Meta's UI looks slightly different from the screenshots in your head — they reshuffle it every few months. Look for the same words even if buttons have moved.

2a. Create the app

1. Go to <https://developers.facebook.com/apps/> and sign in with your Facebook account (the same one linked to your Instagram).
2. **My Apps** → **Create App**.
3. **Use case** → choose **Other** → Next.
4. **App type** → choose **Business** → Next.
5. **Name the app** `ig-notion-sync` (only you see this). **Contact email** = yours.
Business account: select one if you have one, leave blank otherwise.
6. **Create App** → enter your Facebook password if prompted.

You're now looking at the app's dashboard.

2b. Add the Instagram product

1. On the new app's dashboard, scroll down to **Add products to your app** → find **Instagram** → click **Set up**.
2. In the left sidebar you'll now see **Instagram** → **API setup with Instagram login** (the exact wording sometimes changes).
3. Under **Generate access tokens**, click **Add account** and authorize your Instagram Business/Creator account. You'll be bounced through Facebook → Instagram → back to the developer site.
4. Once added, Meta generates a **short-lived access token** (only good for 1 hour). Copy it somewhere temporary — you'll trade it for a 60-day token in step 2d.

2c. Copy the App ID and App Secret

These are like a username and password for your app. The tool uses them to refresh the 60-day token.

1. Left sidebar → **App settings** → **Basic**.
2. Copy the **App ID** — this is your `META_APP_ID`.
3. Click **Show** next to **App secret**, re-enter your Facebook password, copy it. This is your `META_APP_SECRET`. **Treat it like a password** — don't paste it in chat, don't share screenshots of it.

2d. Exchange for a long-lived (60-day) token

The short-lived token from 2b only lasts an hour. We trade it for a 60-day one now so you don't have to redo all of step 2 every few hours.

 **With Claude Code:** Skip this step here. In Step 3's prompt, Claude will run the trade for you after you paste the three values (App ID, App Secret, short-lived token).

■ **Manual:** Open a terminal and run this, replacing each `YOUR_...` with your actual value:

```
curl -sG "https://graph.facebook.com/v21.0/oauth/access_token" \
  -d "grant_type=fb_exchange_token" \
  -d "client_id=YOUR_META_APP_ID" \
  -d "client_secret=YOUR_META_APP_SECRET" \
  -d "fb_exchange_token=YOUR_SHORT_LIVED_TOKEN"
```

The response is a chunk of JSON like

```
{"access_token":"EAA...", "token_type":"bearer", "expires_in":5183944}.
```

Copy the `access_token` value (the long string starting with `EAA`) — this is your `IG_ACCESS_TOKEN`, good for about 60 days.

2e. Find your Instagram User ID

Meta needs to know *which* Instagram account to pull metrics from. You'll find its numeric ID by asking Meta which Facebook Pages you manage, then asking which Instagram account is attached to the right Page.

■ **With Claude Code:** Skip — Claude does this in Step 3's prompt.

■ **Manual:** Open a terminal and run these in order:

```
# 1) List Facebook Pages you manage
curl -s "https://graph.facebook.com/v21.0/me/accounts?
access_token=YOUR_LONG_LIVED_TOKEN"

# 2) From the response, find the `id` of the Page linked to your IG
account, then:
curl -s "https://graph.facebook.com/v21.0/PAGE_ID?
fields=instagram_business_account&access_token=YOUR_LONG_LIVED_TOKEN"
```

The second response includes `"instagram_business_account": {"id": "17841..."}`. The 17-digit number is your `IG_USER_ID`.

By the end of Step 2 you should have collected:

- `META_APP_ID`
- `META_APP_SECRET`
- The short-lived token (Claude path) **or** the long-lived `IG_ACCESS_TOKEN` (manual path)
- `IG_USER_ID` (manual path only; Claude path computes this for you)

Step 3 — Install the tool and configure `.env`

What you're doing and why

Now you've got all the keys. This step downloads the tool to your computer, installs the small Python toolchain it depends on, and writes your keys into a hidden config file called `.env` so the tool can read them.

Some terms you'll see here:

- **uv** — a small program that manages Python for you. You don't have to install Python separately; uv installs the right version automatically the first time you run the tool. (Think of it like a self-contained app store for Python tools.)
- **The tool's code** — a folder of small Python files that does the actual Instagram → Notion work. Claude downloads it for you in one step; you don't need to know where it's coming from or what's inside.
- **.env file** — a plain text file where you put secret values (tokens, IDs). The tool reads it at startup. It never leaves your computer.

You don't need to install Python yourself — uv handles that. You don't need a GitHub account either. You don't need to understand any of this to follow the prompts below.

With Claude Code (recommended)

Open Claude Code (Step 0 explains how). Once you see the Claude prompt, paste this **whole block** as one message:

I'm setting up ig-notion-sync, a tool that pulls Instagram metrics into a Notion database. I'm not technical — please do all the terminal work for me and explain what you're doing in plain English. Do not echo any secrets I give you back into chat — only write them into the `.env` file silently and confirm which fields were written (not their values).

Phase A — install the tool:

1. Check if ``uv`` is installed (``uv --version``). If not, install it:
 - Mac/Linux/WSL: ``curl -LsSf https://astral.sh/uv/install.sh | sh``
 - Make sure ``uv`` is on PATH for this session.
2. Download the tool: ``git clone https://github.com/jgaleria/ig-notion-sync.git ig-notion-sync``. Don't explain git to me — just say "downloading the tool."
3. ``cd ig-notion-sync``
4. ``uv sync``
5. ``cp .env.example .env``
6. Run ``uv run ig-sync --help`` to confirm the install worked.

Phase B — Notion config:

1. Ask me for my Notion integration secret (starts with `ntn_` or `secret_`). Write it to ``.env`` as `NOTION_TOKEN` silently.
2. Ask me for the URL of my Notion database. Extract the 32-character ID before `?v=`, reformat with dashes as `xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx`, write to `.env` as NOTION_DATA_SOURCE_ID.`

Phase C — Instagram config:

1. Ask me for META_APP_ID and META_APP_SECRET. Write to ``.env``.
2. Ask me for the short-lived token I just generated in Meta's dashboard.
3. Run the long-lived token exchange curl using the three values above:
`curl -sG "https://graph.facebook.com/v21.0/oauth/access_token" -d "grant_type=fb_exchange_token" -d "client_id=$META_APP_ID" -d "client_secret=$META_APP_SECRET" -d "fb_exchange_token=$SHORT_LIVED"`
4. Parse the JSON response for ``access_token``, write to ``.env`` as `IG_ACCESS_TOKEN`.
5. Run ``curl -s "https://graph.facebook.com/v21.0/me/accounts?access_token=$LONG_LIVED"`` – list the Pages back to me, ask which one corresponds to my IG account.
6. Take that Page ID and run ``curl -s "https://graph.facebook.com/v21.0/PAGE_ID?fields=instagram_business_account&access_token=$LONG_LIVED"`` – extract the ``instagram_business_account.id``, write to ``.env`` as `IG_USER_ID`.

After all three phases, tell me which fields are in ``.env`` (not their values) and confirm the install looks healthy.

Claude will walk you through it, asking for each value as it needs it.

You'll paste your keys when asked; Claude writes them into ``.env`` for you and never echoes them back.

■ Manual (terminal)

If you'd rather do it yourself, open a terminal:

```
# 1) Install uv (manages Python for you – includes its own Python)
curl -Lsf https://astral.sh/uv/install.sh | sh
# Close this terminal window and open a fresh one so `uv` is on your PATH.

# 2) Download and install
cd ~/Documents/Code # or wherever you keep code; create the folder first
if it doesn't exist
git clone https://github.com/jgaleria/ig-notion-sync.git ig-notion-sync
cd ig-notion-sync
uv sync # this installs Python and the tool's dependencies –
first run can take a minute
cp .env.example .env
```

Now open the ``.env`` file in any text editor (TextEdit, VS Code, Notepad, etc.) and paste your values next to each key:

```
IG_USER_ID=17841... # from step 2e
IG_ACCESS_TOKEN=EAA... # from step 2d
META_APP_ID=... # from step 2c
META_APP_SECRET=... # from step 2c
NOTION_TOKEN=ntn... # from step 1b
```

```
NOTION_DATA_SOURCE_ID=xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxxx # from step
1d
```

Leave `IG_GRAPH_API_VERSION`, `NOTION_VERSION`, `DRY_RUN`, and `MAX_POSTS` at their defaults. Save the file.

Step 4 — First sync

What you're doing and why

Now you actually pull your Instagram posts into Notion. To stay safe, you'll do it in three passes:

1. **Dry run** — the tool reads everything from Instagram and *describes* what it would write to Notion, without actually writing. This is a safety check.
2. **Single-post test** — write just one post, so you can eyeball the result in Notion before committing to the full sync.
3. **Full sync** — pull everything in.

You can re-run the full sync any time you want fresh numbers. It's safe to re-run — the tool matches posts by Instagram ID and just refreshes the metrics on existing rows.

With Claude Code

In the same Claude Code session you used for Step 3, paste:

```
Run my first ig-notion-sync. Walk me through it:
1. Run `uv run ig-sync --dry-run` and show me the plan. Explain in plain
English what would be created vs updated. Warn me if anything looks
suspicious (wrong account, zero posts found).
2. If the plan looks reasonable, ask me to confirm.
3. Run `uv run ig-sync --limit 1` — a single real write to test.
4. Tell me to open my Notion database and check the new row.
5. Once I confirm, run the full `uv run ig-sync` for all posts.
6. Read `logs/last_run.json` and summarize: how many created, updated, any
errors.
```

```
If anything fails, read the error and tell me in plain English what's
wrong and which earlier step probably caused it.
```

Manual

In the terminal, inside the `ig-notion-sync` folder:

```
uv run ig-sync --dry-run      # preview only, no writes
uv run ig-sync --limit 1     # write one post as a test
```

```
uv run ig-sync # full sync
```

Open your Notion database — confirm rows appeared with metrics filled in.

Step 5 — Keep it running

What you're doing and why

Two things to know about long-term operation:

1. **The Instagram token expires every 60 days.** When that happens the tool stops working until you refresh it. The refresh is one curl command, and Claude can do it for you.
2. **The tool is one-shot.** It doesn't sit there running — every time you want fresh metrics, you (or a scheduler) have to run it. If you want it automatic, you set up a daily scheduler (Mac, Linux, and Windows each have their own).

Token refresh (every ~60 days)

Your `IG_ACCESS_TOKEN` expires. When it does, the tool exits with `Access token rejected (code 190)`.

With Claude Code:

```
My IG access token expired. Re-run the Meta long-lived token exchange curl using the current META_APP_ID, META_APP_SECRET, and IG_ACCESS_TOKEN from .env as the fb_exchange_token. Write the new token back into .env. Then run `uv run ig-sync --dry-run` to confirm.
```

 **Manual:** re-run the curl from step 2d using your *current* token as `fb_exchange_token`. Meta returns a fresh 60-day token. Paste into `.env`.

Set a calendar reminder for day 50 so the token doesn't lapse.

Schedule it (optional)

The tool is one-shot. To run it automatically every day:

With Claude Code:

```
Set up a daily schedule for ig-notion-sync. Detect my OS, then set up the right scheduler:  
- Mac: a launchd .plist in ~/Library/LaunchAgents  
- Linux: a crontab entry  
- Windows: explain that I'll need Task Scheduler manually  
The command should be `uv run --directory <full path to ig-notion-
```

```
sync> ig-sync` with output piped to logs/cron.log. Test it fires once.
```

■ **Manual:** add a daily cron entry / launchd plist / Task Scheduler job that runs `uv run --directory /full/path/to/ig-notion-sync ig-sync`.

Alternative: a GitHub Actions workflow on a schedule. Put your env values in the repo's Actions secrets — *don't* commit `.env`.

Step 6 — Ask Claude about your analytics

What you're doing and why

Once your Notion database fills up with metrics, the natural next question is "so what?" Instead of staring at columns of numbers, you can ask **Claude** (the chat app at claude.ai) to read your database directly and answer questions in plain English: *what were my top posts last month, which Reels had the best watch time, what do my best-performing captions have in common.*

(Note: the Claude chat app at claude.ai is a different thing from Claude Code, which is the terminal helper you used in earlier steps. For this step you want the regular chat app — in your browser, or the Claude desktop or mobile app.)

Two ways to give Claude your data

Option A — Export as CSV and drop it into Claude (RECOMMENDED). This is the default. Every time you want fresh analysis, you export your Notion database as a CSV file and attach it to a chat. Why it's preferred: the CSV contains **every published row in one shot**, so when Claude filters and ranks, the result is provably complete. Downside: it's a snapshot — re-export each time the numbers update.

Option B — Connect Claude to your Notion workspace (fallback). A one-time setup. After it's done, Claude can read your database without you attaching anything, and it can write back too (e.g. for the tagging prompt below). **But:** the Notion connector has no "list all rows" call — it searches by keyword and returns at most ~25 results per call. That means for ranking-style analytics ("top posts by reach"), Claude has to seed keyword passes and hope it covers everything. **You can't prove completeness through the connector**, and a post whose caption missed your seed words gets silently dropped. Use the connector for live spot-checks (current numbers on a few named posts) or for writeback, not for full-DB ranking.

Default to Option A. Add Option B only if you also want the tagging prompt or live spot-checks.

Option A — Export the database as CSV

Do this every time you want fresh numbers:

1. Run a sync first (`uv run ig-sync`) so Notion has the latest metrics.
2. Open your Notion database as a full page.
3. Click the `...` menu in the top-right → **Export**.
4. **Export format:** Markdown & CSV. Click **Export**. Notion downloads a `.zip`.
5. Unzip the file (double-click on Mac; right-click → Extract on Windows).
Inside is a `.CSV` named after your database.
6. Open a new chat at claude.ai. Drag the `.CSV` into the chat window (or click the paperclip / attach button and pick it).
7. Paste one of the analytics prompts below in the same message.

The free plan handles this fine for typical creator databases (hundreds of rows).

Option B — Connect Claude to Notion (one-time setup)

1. Sign in at claude.ai (free plan works for light use; Pro lets you do more in one conversation).
2. Open Claude's **Settings** → look for the section for **Connectors / Integrations / Tools** (Anthropic occasionally renames it).
3. Find **Notion** in the list and click **Connect**. You'll be sent to Notion to authorize Claude to read your workspace.
4. Make sure the database you built in Step 1 is in the same workspace you just authorized. (It is, if you've only got one.)

Once connected, you don't need to attach anything — just paste the prompts below and Claude will pull the data live.

Copy-paste prompt — analytics runbook

Open a fresh chat with Claude. If you're on Option A, **attach the CSV first in the same message**. Then paste this whole block. Edit the **QUESTION** section at the bottom to change the metric, window, or tie-break for this particular run.

Before first use: fill in **your own content pillars** in the COVERAGE PROTOCOL section (the bracketed list). These are the recurring themes you post about — e.g. fitness, mindset, building/coding, lifestyle. They're only used if Claude falls back to the connector path.

Analyze my Instagram metrics from my "Content" Notion database.

DATA SOURCE — try in this order:

1. CSV (PREFERRED). If a CSV export of the Content DB is attached in this chat, use it. It guarantees full coverage — every published row is in there, so filtering and ranking are provably complete. Read it, filter, rank — done.

2. Notion connector (FALLBACK). Only use this if no CSV is attached. If you do, you MUST follow the coverage protocol below, because connector search cannot prove completeness.

FINDING THE DB (connector path only):

- Find the database named exactly "Content" (NOT "Content Inspo", "Content Production", or any similar variant).
- Status values are: Ideas for later, Idea, Draft, Record, Editing, Done. ONLY rows with Status = "Done" carry metrics – everything else is an empty scripting template. Skip them.

COVERAGE PROTOCOL (connector path only – CSV skips this entirely):

- The Notion connector has no "list all rows" call; search returns ~25 keyword-ranked results, and only titles/snippets – not metric values. Metrics require opening each candidate page individually with fetch.
- Run MULTIPLE keyword passes seeded from my content pillars, merge + dedupe candidates, and keep going until a pass returns NO new posts (saturation). My pillars: [REPLACE WITH YOUR PILLARS – e.g. fitness / gym / lifting, mindset / discipline / motivation, building / coding / software, lifestyle / vlog / daily, layoff / unemployment / career]. Add adjacent angles (gear, family, city, etc.) until saturation.
- Open each candidate page; keep ONLY Status = "Done" rows.

DEFAULTS (override in the QUESTION section below):

- Metric = Reach
- Window = published on/after (latest Last Synced date – 30 days)
- Tie-break = Total views

OUTPUT (in this order):

1. A markdown table FIRST, sorted by [Metric] desc, tie-break [Tie-break]:
Post name, Date, [Metric], Total views, Likes, Comments, Saves, Shares, Avg Watch (s).
2. Then ≤200 words: what the top performers share, and anything surprising or actionable. Don't restate the table.
3. Flag any post within ~2 days of the window edge whose inclusion/exclusion would reorder the ranking.
4. Report: the published-row count, which data source you used (CSV or connector), and – if connector – explicitly note that coverage is keyword-bounded and not provably complete.

QUESTION (edit each run):

```
Metric      = [default: Reach]
Window      = [default: latest Last Synced – 30 days]
Tie-break   = [default: Total views]
```

For one-off exploratory questions outside this ranking format (e.g.

"compare Reels vs Carousels on saves-per-view", "which day of week performs best", "summarize this month vs last"), just attach the CSV and ask in plain English — the structured runbook above is only needed when ranking matters.

Copy-paste prompt — auto-tag the **Topics** column

The sync intentionally leaves **Topics** blank so you can categorize manually. But you can hand that off to Claude too. **This one requires Option B** (the Notion connector) because Claude needs to write the tags back into Notion. Paste this in a fresh chat:

```
I have a Notion database called "Content" that tracks my Instagram posts. Each row has a `Caption` column with the IG caption and a `Topics` column where I categorize posts by theme.

Please:
1. Use the Notion connector to find all rows where `Topics` is empty AND `Platform` is "Instagram".
2. For each one, read the `Caption` and suggest a Topic based on the existing Topic values already used in other rows in this database. Reuse my existing taxonomy – don't invent new tags unless absolutely nothing fits.
3. List your suggestions back to me as a table: post Name → suggested Topic → one-line reason.
4. WAIT for me to confirm before writing anything back to Notion. Don't update any rows until I say go.

Do not touch the `Name`, `Mission`, `Intensity`, or `Inspo` columns under any circumstances.
```

Tips

- **Run a sync first** so the numbers Claude reads are current. Stale data leads to confident but wrong answers.
- **If you exported a CSV but Claude says it can't read it**, try uploading again — occasionally the attachment fails silently. The file should appear as a chip above the message.
- **If you're on the connector and Claude can't find the database**, double-check the Notion connector shows as connected in Settings, and try referring to the database by its exact title.
- **Claude reads, then asks before writing.** You're always in the loop — no rows get updated unless you confirm in chat.
- **Long analyses eat tokens.** If a free-plan conversation runs out, start a new chat and narrow the question.

Troubleshooting

Symptom	Likely cause	Fix
ValidationError: IG_USER_ID Field required	.env missing or empty	Re-check Step 3
Access token rejected (code 190)	IG token expired	Re-run Step 2d's curl with current token

Symptom	Likely cause	Fix
<code>object_not_found</code> from Notion	Integration not connected to DB	Step 1c
<code>validation_error: ... is not a property</code>	Notion column missing/misspelled	Step 1a's schema table
Insights unavailable for media X	Post < 30 min old	Script handles it; re-run later
<code>uv: command not found</code>	New terminal not opened after install	Close terminal, open a fresh one
Thumbnails 404 after a week	IG CDN URLs expire	Re-run sync to refresh
Sync completes but Notion shows nothing	Wrong workspace / wrong database	Confirm <code>NOTION_DATA_SOURCE_ID</code> matches the DB you opened

If Claude gets confused or stuck mid-prompt, paste:

```
Stop. Re-read SETUP.md in this folder and tell me which step we're on and what to do next.
```

Still stuck?

Some setups hit snags no guide can fix — a Meta account flagged for review, an Instagram account that won't switch to Business, a Notion workspace with unusual permissions. If you've been at it for more than 90 minutes, feel free to reach out.